Mike 后端任务 SOP

> Austin 已完成后端基础骨架，本文档是你（Mike）剩余任务的完整作业说明。
> 每完成一个小任务就 commit 一次，分支：`feature-mike-strapi-controllers`

任务总览

优先级	任务	预计时间
🔴 P0	初始化 Strapi 项目（package.json + 依赖）	1 天
🔴 P0	配置环境变量（.env 文件）	0.5 天
🔴 P0	实现 6 个 Content Type 的权限配置	0.5 天
🟡 P1	实现 Strapi Controllers（含业务逻辑扩展）	2 天
🟡 P1	初始化 NestJS 微服务依赖	0.5 天
🟡 P1	实现 `registrationCountByEvent` GraphQL resolver	1 天
🟢 P2	实现 export.processor.ts（服务端 CSV 导出）	1 天
🟢 P2	接入真实邮件发送（email.processor.ts）	1 天
🟢 P2	写种子数据（测试用的假数据）	0.5 天

Task 1 — 初始化 Strapi 项目 🔴

背景

Austin 只写了 Strapi 的配置文件和 schema，但没有 `package.json` 和项目初始化。
你需要用 Strapi CLI 创建项目结构，然后把 Austin 写的文件覆盖进去。

步骤

第一步：进入 strapi 目录

```
cd backend/strapi
```

第二步：检查 `package.json` 是否存在

```
ls
```

如果没有 `package.json`，执行第三步。如果有，跳到第五步。

第三步：用 Strapi CLI 初始化（选 TypeScript）

```
# 在 backend/ 目录运行（不是 backend/strapi/）
cd ..
npx create-strapi-app@latest strapi --no-run --ts
# 选项选择：
#   Installation type: Custom
#   Database: PostgreSQL
#   TypeScript: Yes
```

第四步：把 Austin 的文件覆盖回去

Strapi CLI 会生成新的 config 和 src/api 文件，需要把 Austin 写的版本复制回来：

```
# Austin 的配置文件已在 git 里
# 用 git 恢复会自动覆盖 CLI 生成的模板
git checkout backend/strapi/config/
git checkout backend/strapi/src/api/
git checkout backend/strapi/src/extensions/
```

第五步：安装依赖

```
cd backend/strapi
npm install

# 安装 PostgreSQL 驱动（Strapi 默认不带）
npm install pg
```

第六步：验证

```
npm run develop
# 访问 http://localhost:1337/admin
# 应该看到 Strapi 首次初始化界面（设置 admin 账户）
```

Commit：

```
feat(strapi): 初始化 Strapi 项目并安装 PostgreSQL 驱动
```

Task 2 — 配置环境变量 🔴

背景

所有数据库密码、API Token、Redis 地址都要通过 `.env` 文件配置，不能写死在代码里。

步骤

第一步：在 `backend/strapi/` 下创建 `.env` 文件

```
# 在 backend/strapi/.env 写入以下内容:
HOST=0.0.0.0
PORT=1337
APP_KEYS=your-generated-key-1,your-generated-key-2
API_TOKEN_SALT=your-api-token-salt
ADMIN_JWT_SECRET=your-admin-jwt-secret
TRANSFER_TOKEN_SALT=your-transfer-token-salt
JWT_SECRET=your-jwt-secret

# 数据库（对应 Docker Compose 的配置）
DATABASE_CLIENT=postgres
DATABASE_HOST=localhost
DATABASE_PORT=5432
DATABASE_NAME=cssa_web_dev
DATABASE_USERNAME=postgres
DATABASE_PASSWORD=postgres
DATABASE_SSL=false
```

生成随机 key（复制到上面的 your-xxx 位置）：

```
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"
# 运行 5 次，分别填入 APP_KEYS（两个）、API_TOKEN_SALT、ADMIN_JWT_SECRET、
TRANSFER_TOKEN_SALT
```

第二步：在 `backend/microservice/` 下创建 `.env` 文件

```
# 在 backend/microservice/.env 写入:
PORT=3002
NODE_ENV=development

# Strapi 连接
STRAPI_URL=http://localhost:1337
STRAPI_API_TOKEN=（下一步从 Strapi Admin 获取）
```

```
# Redis 连接（对应 Docker Compose）
REDIS_HOST=localhost
REDIS_PORT=6379

# 前端域名（CORS）
CLIENT_URL=http://localhost:3000
ADMIN_FRONTEND_URL=http://localhost:3001
```

第三步：获取 Strapi API Token

1. 启动 Strapi：`npm run develop`
2. 进入 `http://localhost:1337/admin`
3. 左侧菜单 → Settings → API Tokens → Create new API Token
4. Name：`microservice-full-access`
5. Token type：Full Access
6. 点击 Save，复制生成的 Token
7. 填入 `backend/microservice/.env` 的 `STRAPI_API_TOKEN=`

第四步：确认 .gitignore 包含 .env

```
cat backend/strapi/.gitignore | grep ".env"
# 应该能看到 .env 在 ignore 列表里
# 如果没有，手动添加：
echo ".env" >> backend/strapi/.gitignore
echo ".env" >> backend/microservice/.gitignore
```

> ⚠️ .env 文件绝对不能提交到 git！里面有数据库密码和 Token。

Commit（只提交 .gitignore 更新，不提交 .env）：

```
chore: 更新 .gitignore 确保 .env 不被提交
```

Task 3 — 配置 API 权限 🔴

背景

Strapi 默认所有接口都是私有的（需要登录），需要手动配置哪些接口是公开的。

步骤

第一步：启动 Strapi，进入 Admin

```
http://localhost:1337/admin
```

第二步：进入权限设置

```
Settings → Users & Permissions Plugin → Roles → Public
```

第三步：按照下表勾选权限

Content Type	公开操作（Public 勾选）	私有操作（Authenticated 才能用）
Site-config	find	update
Department	find, findOne	create, update, delete
Member	find, findOne	create, update, delete
Event	find, findOne	create, update, delete
Past-event	find, findOne	create, update, delete
Registration	**create**	find, findOne, update, delete

> 特别注意：Registration 的 create 必须是 Public（前台用户提交报名），
> 但 find/update/delete 必须是 Authenticated（防止用户看到别人的报名信息）。

第四步：保存并验证

```
# 验证公开接口可以访问（不带 Token）
curl http://localhost:1337/api/events
# 应该返回活动数据，不是 403

# 验证受保护接口被拦截（不带 Token）
curl http://localhost:1337/api/registrations
# 应该返回 403 Forbidden
```

Commit：

```
# 权限配置是在 Strapi Admin UI 里点的，不产生代码改动
# 但可以导出配置文件提交
docs(strapi): 记录 API 权限配置说明
```

Task 4 — 实现 Strapi Controllers 🟡

背景

Austin 在每个 Content Type 下都留了 `controllers/*.ts` 文件，目前是 `export {}`（空的）。Strapi 默认的 Controller 已经能处理基础 CRUD，但有几个需要自定义业务逻辑的地方。

4.1 Registration Controller（最重要）

文件： `backend/strapi/src/api/registration/controllers/registration.ts`

你需要实现：

```
import { factories } from '@strapi/strapi'

// 继承 Strapi 默认 Controller，并扩展 create 和 update 方法
export default factories.createCoreController(
  'api::registration.registration',
  ({ strapi }) => ({

    // 覆盖 create：校验活动状态和容量
    async create(ctx) {
      const { event: eventId, user_info } = ctx.request.body.data

      // 第一步：查询活动
      const event = await strapi.entityService.findOne(
        'api::event.event',
        eventId,
        { populate: ['registrations'] }
      )

      if (!event) {
        return ctx.badRequest('活动不存在')
      }

      // 第二步：检查活动状态
      if (event.status !== 'active') {
        return ctx.badRequest('报名通道已关闭，该活动当前不接受报名')
      }

      // 第三步：检查容量（capacity 为 null 表示不限人数）
      if (event.capacity !== null) {
        const count = await strapi.entityService.count(
          'api::registration.registration',
          {
            filters: {
```

```
              event: { id: eventId },
              status: { $ne: 'cancelled' },  // 已取消的不算
            },
          }
        )
        if (count >= event.capacity) {
          return ctx.badRequest(`报名人数已满（上限 ${event.capacity} 人）`)
        }
      }

      // 第四步：校验通过，调用默认 create 写入数据库
      const response = await super.create(ctx)
      return response
    },

    // 覆盖 update：限制只能改 status 字段（防止管理员意外改动 user_info）
    async update(ctx) {
      const allowedFields = ['status']
      const bodyKeys = Object.keys(ctx.request.body.data || {})
      const invalidFields = bodyKeys.filter(k => !allowedFields.includes(k))

      if (invalidFields.length > 0) {
        return ctx.badRequest(
          `不允许修改字段：${invalidFields.join(', ')}，只能修改 status`
        )
      }

      return super.update(ctx)
    },
  })
)
```

验证：

```
# 测试：向 closed 活动提交报名（应该返回 400）
curl -X POST http://localhost:1337/api/registrations \
  -H "Content-Type: application/json" \
  -d '{"data":{"event":1,"user_info":{"name":"test"},"status":"pending"}}'
# 如果活动 status 是 closed，应该返回：
# { "error": { "message": "报名通道已关闭" } }
```

Commit：

```
feat(strapi): 实现 Registration Controller 业务校验逻辑
```

4.2 Event Controller（可选，添加报名人数统计）

文件： `backend/strapi/src/api/event/controllers/event.ts`

```
import { factories } from '@strapi/strapi'

export default factories.createCoreController(
  'api::event.event',
  ({ strapi }) => ({

    // 覆盖 find：在每个活动里追加 registrationCount 字段
    async find(ctx) {
      const { data, meta } = await super.find(ctx)

      // 为每个活动追加当前报名人数
      const enriched = await Promise.all(
        data.map(async (event) => {
          const count = await strapi.entityService.count(
            'api::registration.registration',
            {
              filters: {
                event: { id: event.id },
                status: { $ne: 'cancelled' },
              },
            }
          )
          return {
            ...event,
            attributes: {
              ...event.attributes,
              registrationCount: count,    // 追加字段
            },
          }
        })
      )

      return { data: enriched, meta }
    },
  })
)
```

Commit：

```
feat(strapi): Event Controller 追加报名人数统计字段
```

4.3 其他 Controller（使用默认即可）

Department、Member、Site-config、Past-event 这四个 Controller 不需要自定义逻辑，
使用 Strapi 默认的 Controller 就够用。

直接在各自的 `controllers/*.ts` 文件里写：

```
// backend/strapi/src/api/department/controllers/department.ts
import { factories } from '@strapi/strapi'
export default factories.createCoreController('api::department.department')

// backend/strapi/src/api/member/controllers/member.ts
import { factories } from '@strapi/strapi'
export default factories.createCoreController('api::member.member')

// backend/strapi/src/api/past-event/controllers/past-event.ts
import { factories } from '@strapi/strapi'
export default factories.createCoreController('api::past-event.past-event')
```

Commit：

```
feat(strapi): 实现 Department / Member / Past-event / Site-config 默认
Controller
```

Task 5 — 初始化 NestJS 微服务依赖 🟡

背景

Austin 写了微服务的代码，但没有 `package.json`，需要初始化 NestJS 项目并安装依赖。

步骤

第一步：进入 microservice 目录检查

```
cd backend/microservice
ls
```

```
# 如果没有 package.json，继续下面步骤
```

第二步：初始化 NestJS 项目

```
# 在 backend/ 目录运行
cd ..
npx @nestjs/cli@latest new microservice --package-manager npm
# 选 npm，不选 yarn
```

第三步：安装必要依赖

```
cd backend/microservice

# NestJS 核心依赖
npm install @nestjs/config @nestjs/bull bull ioredis

# 请求校验（DTO 用）
npm install class-validator class-transformer

# HTTP 客户端（调 Strapi API 用）
npm install @nestjs/axios axios

# TypeScript 类型
npm install -D @types/bull
```

第四步：把 Austin 的代码恢复

```
git checkout backend/microservice/src/
```

第五步：验证启动

```
npm run start:dev
# 终端应看到：微服务启动成功，端口：3002
```

Commit：

```
feat(microservice): 初始化 NestJS 项目并安装依赖
```

Task 6 — 实现 GraphQL registrationCountByEvent Resolver 🟡

背景

Austin 在 `schema.graphql` 里定义了这个 Query 的签名，但 resolver（实际执行逻辑）还没写。

步骤

第一步：在 Strapi extensions 里创建 resolver 文件

```
# 创建文件
touch backend/strapi/src/extensions/graphql/resolvers.ts
```

第二步：实现 resolver

```
// backend/strapi/src/extensions/graphql/resolvers.ts
export default {
  Query: {
    // 对应 schema.graphql 里定义的 registrationCountByEvent(eventId: ID!):
Int
    registrationCountByEvent: async (
      _parent: unknown,
      args: { eventId: string },
      context: { strapi: typeof strapi }
    ) => {
      const { eventId } = args

      // 统计该活动的有效报名数（不含已取消）
      const count = await context.strapi.entityService.count(
        'api::registration.registration',
        {
          filters: {
            event: { id: eventId },
            status: { $ne: 'cancelled' },
          },
        }
      )

      return count
    },
  },
}
```

第三步：在 Strapi 入口注册 resolver

```
// backend/strapi/src/index.ts（新建文件）
import resolvers from './extensions/graphql/resolvers'

export default {
  register({ strapi }) {
    // 注册自定义 GraphQL resolver
    const extensionService = strapi.plugin('graphql').service('extension')
    extensionService.use({
      resolvers,
    })
  },
}
```

第四步：验证

在 GraphQL Playground 里运行：

```
query {
  registrationCountByEvent(eventId: "1")
}
```

应该返回一个数字。

Commit：

```
feat(strapi): 实现 registrationCountByEvent GraphQL resolver
```

Task 7 — 写种子数据（Seed Data）🟢

背景

Emilia 和 Yeon 开发时需要有真实数据才能调试 UI，需要你手动在 Strapi Admin 里创建测试数据。

需要创建的测试数据

全站配置（1 条）：

- hero_title：欢迎来到 UTMCSSA

- hero_subtitle：多伦多大学密西沙加校区中国学生学者联合会
- contact_email：utmcssa@gmail.com
- instagram_url：https://instagram.com/utmcssa

部门（至少 3 个）：

- 主席团 / 市场部 / 学术部

成员（每个部门至少 2 个）：

- 随便填姓名、职位，上传任意图片

活动（2 个）：

- 1 个 `active`（开放报名）：
  - form_schema 至少包含姓名（text）、邮箱（email）、年级（select）
- 1 个 `upcoming`（待开始）

往期活动（3 个）：

- 不同日期，上传任意图片

Task 8 — 接入邮件发送 🟢

背景

`email.processor.ts` 目前只打日志，需要接入真实的 SMTP 邮件发送。

步骤

第一步：安装 Nodemailer

```
cd backend/microservice
npm install nodemailer @types/nodemailer
```

第二步：在 `.env` 里添加 SMTP 配置

开发阶段推荐使用 Mailtrap（免费，邮件不会真发出去，可以在 Web 界面看到）：

```
SMTP_HOST=sandbox.smtp.mailtrap.io
SMTP_PORT=2525
SMTP_USER=你的Mailtrap用户名
```

```
SMTP_PASS=你的Mailtrap密码
EMAIL_FROM=noreply@utmcssa.com
```

第三步：更新 email.processor.ts

```
import * as nodemailer from 'nodemailer'

const transporter = nodemailer.createTransport({
  host: process.env.SMTP_HOST,
  port: Number(process.env.SMTP_PORT),
  auth: { user: process.env.SMTP_USER, pass: process.env.SMTP_PASS },
})

@Process('registration_confirmed')
async handleRegistrationConfirmed(job: Job<EmailJobData>) {
  const { userEmail, eventTitle } = job.data

  await transporter.sendMail({
    from: process.env.EMAIL_FROM,
    to: userEmail,
    subject: `【UTMCSSA】${eventTitle} 报名成功`,
    html: `
      <h2>报名成功！</h2>
      <p>您已成功报名 <strong>${eventTitle}</strong>。</p>
      <p>我们将在审核后通过邮件通知您结果。</p>
    `,
  })

  this.logger.log(`确认邮件已发送至 ${userEmail}`)
}
```

Commit：

```
feat(microservice): 接入 Nodemailer 实现报名确认邮件发送
```

完成检查清单

在告诉 Austin「我做完了」之前，请逐项确认：

- [ ] `docker-compose up -d` 能启动 PostgreSQL 和 Redis（两个都 healthy）
- [ ] `cd backend/strapi && npm run develop` 能启动，访问 `http://localhost:1337/admin`

- [ ] Strapi Admin 里能看到 6 个 Content Type，字段和文档一致
- [ ] API 权限配置完成（公开接口无需 Token，受保护接口返回 403）
- [ ] Registration Controller 实现了活动状态和容量校验
- [ ] `cd backend/microservice && npm run start:dev` 能启动
- [ ] POST /api/v1/registrations 测试成功（见 Task 4 的 curl 命令）
- [ ] 速率限制测试：连发 6 次，第 6 次返回 429
- [ ] GraphQL Playground 里能查询 departments、events、site-config
- [ ] `registrationCountByEvent` GraphQL query 返回正确数字
- [ ] Strapi 里有足够的测试数据供 Emilia/Yeon 联调

> 遇到问题：先查本文档 → 查接口文档 → 在群里 @ Austin
> 不要自己扛超过 30 分钟没有进展